# جزوه‌ی هفته‌ی ۵: مهندسی داده برای داده‌های حسگر نویزی

داده‌ی گمشده، داده‌ی پرت، نرمال‌سازی و تقسیم بدون نشت روی لاگ یک ربات

هوش مصنوعی برای مهندسی هوافضا و مکانیک (دکتری)، بخش صفر

---

> **منشأ محتوا:** بخشی بسط‌یافته از جزوه‌ی مدرس. بخش ۱ اسلایدهای ۲ تا ۸ ارائه‌ی مدرس «یادگیری ماشین و درختان تصمیم‌گیری» را بسط می‌دهد (هوش مصنوعی ⊃ یادگیری ماشین ⊃ یادگیری عمیق؛ استخراج ویژگی؛ ناتوانی روش‌های کلاسیک در کار مستقیم با فایل CSV خام؛ دسته‌بندی، خوشه‌بندی و رگرسیون). بقیه محتوای جدید است. سازوکارهای گمشدگی از R. J. A. Little و D. B. Rubin, *Statistical Analysis with Missing Data* (2019), Ch. 1؛ پاک‌سازی، جایگذاری، قاعده‌های داده‌ی پرت، مقیاس‌بندی و نشت داده با بازنویسی (نه رونویسی) از W3Schools (Pandas - Cleaning Data، ML Scaling، ML Train/Test) و GeeksforGeeks (Handling Missing Values in Machine Learning، Detect and Remove the Outliers using Python، Data Leakage)؛ انواع خطای LiDAR از Thrun، Burgard و Fox, *Probabilistic Robotics* (2005), Sec. 6.3؛ کران امتیاز z از Samuelson, JASA 63 (1968)؛ فیلتر Hampel از Pearson (2002)؛ و اعتبارسنجی بلوکی از Roberts et al., *Ecography* 40 (2017).

## ۱. مسئله

ربات پروژه (ماژول M3) باید پیش از دیوار ترمز کند. کنترل‌گر آن فاصله‌ی LiDAR را **یک ثانیه بعد**، $r(t + 1\,\mathrm{s})$، از فاصله‌ی فعلی $r$، سرعت چرخ $v$، شتاب طولی $a$ و نرخ گردش $\omega$ لازم دارد. ۱۶ آزمون رانندگی با ۱۰ هرتز در `data/sample/week05_rover_log.csv` (۲۵۳۳ سطر) ثبت شده است. مسیر ساده‌لوحانه این است: خواندن CSV، جابه‌جایی فاصله به اندازه‌ی ۱۰ سطر، حذف سطرهای NaN، تقسیم تصادفی سطرها و برازش. مدل خطی آنگاه **2.73 m** خطا دارد، برای فاصله‌ی توقف یک‌متری. ایراد از مدل نیست. لاگ همه‌ی نقص‌های یک ربات واقعی را دارد و مسیر ساده‌لوحانه هر کدام را غلط می‌خواند: ۴۸۹ صفر ابتدایی LiDAR (پژواکی نیامده، دیوار دورتر از ۱۲ متر) «دیوار چسبیده به حسگر» خوانده می‌شود؛ ۴۱ افت تصادفی و ۱۱ پژواک گردوغبار فاصله‌ی معتبر به نظر می‌رسند؛ ۱۹ سرعت NaN سطرها را بی‌صدا حذف می‌کند؛ ۱۱ جهش شتاب تا $\pm 25$ m/s² مانور به نظر می‌رسد؛ دو بخش سه‌ثانیه‌ای ژیروسکوپ یخ‌زده نرخ گردش بی‌نقصی می‌سازد؛ ۲۵ سطر تکراری داده‌ی آموزش اضافی می‌شود؛ و `shift(-10)` روی رانندگی‌های پشت‌سرهم، ده هدف آخر هر اجرا را از اجرای بعدی برمی‌دارد.

ارائه‌ی مدرس اصل کلی را بیان می‌کند: روش‌های کلاسیک (درخت تصمیم، SVM، بیز ساده، رگرسیون لجستیک) روی CSV خام کار نمی‌کنند و ویژگی آماده لازم دارند؛ حتی شبکه‌های عمیق که ویژگی را خودشان می‌آموزند، داده‌ای می‌خواهند که همان چیزی را بگوید که به نظر می‌رسد. این هفته **رگرسیون** نظارت‌شده است و برچسب از خود لاگ می‌آید.

## ۲. پیش از پاک‌سازی، ممیزی

`audit_frame(frame, sentinels={"lidar_range_m": 0.0})` برای هر ستون تعداد خانه‌های NaN، ±inf و مقدار نگهبان (sentinel)، طولانی‌ترین دنباله‌ی NaN و طولانی‌ترین دنباله‌ی مقادیر **یکسان** را می‌شمارد: با نویز 0.3 °/s ژیروسکوپ، ۳۲ خوانش برابر نمی‌تواند فیزیک باشد، هرچند ۵۱ سرعت صفر فقط یعنی ربات ایستاده است. W3Schools چهار نوع داده‌ی بد را برمی‌شمارد (خانه‌ی خالی، قالب نادرست، داده‌ی نادرست، تکراری)؛ لاگ حسگر دو نوع دیگر هم دارد: **مقدار نگهبان** و **مقدار یخ‌زده**.

## ۳. داده‌ی گمشده: سازوکار تعیین می‌کند

Little و Rubin رده‌بندی را بر اساس **چرایی** گمشدگی انجام می‌دهند:

- **MCAR** (کاملاً تصادفی): به هیچ چیز وابسته نیست. بسته‌ی CAN گمشده.
- **MAR** (تصادفی): به ستون *مشاهده‌شده‌ی* دیگری وابسته است. قطع GPS وقتی پرچم ثبت‌شده‌ی `in_hangar` روشن است.
- **MNAR** (غیرتصادفی): به خود مقدار گمشده وابسته است. LiDAR دقیقاً وقتی ساکت است که دیوار دور است.

### مثال حل‌شده‌ی ۱: جایگذاری میانگین در حالت MNAR

فاصله‌های واقعی در یک رانندگی 2، 4، 6، 8، 10، 13، 15 و 17 متر است و برد LiDAR ۱۲ متر. میانگین واقعی $75/8 = 9.375$ m است. فقط پنج مقدار اول دیده می‌شوند، با میانگین $30/5 = 6.0$ m. پرکردن سه شکاف با 6.0 خطاهای $-7$، $-9$ و $-11$ متر می‌دهد و میانگین پس از جایگذاری همان 6.0 می‌ماند: سوگیری $-3.4$ m که با هیچ مقدار داده‌ای از بین نمی‌رود. در لاگ ربات میانگین واقعی 6.86 m و میانگین مشاهده‌شده حدود 5.1 m است. اطلاعات صادق «دورتر از ۱۲ متر» است: آن را به‌صورت ستون نشانگر نگه دارید یا دامنه‌ی مدل را به برد حسگر محدود کنید، که دستورالعمل کلاس همین کار را می‌کند.

راه‌حل‌ها: حذف سطر (MCAR، با سطرهای کم)؛ درون‌یابی زمانی (شکاف‌های کوتاه داخلی، اینجا حداکثر ۵ نمونه)؛ جایگذاری میانه به‌همراه نشانگر `_was_missing` (ویژگی‌های ضعیف، MAR یا MNAR)؛ محدود کردن دامنه (MNAR ناشی از حد حسگر). هرگز روی شکاف بلند یا میان دو اجرا درون‌یابی نکنید. `MarkInvalid` ابتدا مقدار نگهبان و هر مقدار بیرون از $[0.3, 12]$ متر را NaN می‌کند تا هر راه‌حل «نامعلوم» ببیند، نه «صفر متر».

### مثال حل‌شده‌ی ۲: درون‌یابی یک شکاف

با $r(0) = 8.0$ m و $r(0.3) = 7.4$ m و دو نمونه‌ی گمشده در میان:

$$r(t) = 8.0 + (7.4 - 8.0)\,\frac{t}{0.3}, \qquad r(0.1) = 7.8\ \text{m}, \quad r(0.2) = 7.6\ \text{m}.$$

`FillGaps(max_gap=5, time_column="time_s")` هر دو را بازتولید می‌کند و شکاف‌های بلندتر را NaN نگه می‌دارد. با ۱۰ هرتز و سرعت کمتر از 2.2 m/s، شکاف 0.5 ثانیه‌ای چند سانتی‌متر بیشتر خطا ندارد، مگر ربات درون شکاف ترمز کند.

## ۴. داده‌ی پرت: قاعده‌ی سراسری در برابر محلی

قاعده‌ی z نمونه‌ای را علامت می‌زند که $|x - \bar{x}|/\sigma > 3$. در نمونه‌ی کوچک کار نمی‌کند: با انحراف معیار جامعه، **هر** نقطه در $|z_i| \le \sqrt{n-1}$ صدق می‌کند (Samuelson، 1968)، پس برای $n \le 9$ هیچ نقطه‌ای از ۳ نمی‌گذرد. با انحراف معیار نمونه کران $(n-1)/\sqrt{n}$ است (Shiffler، 1988). داده‌ی پرت همان $\sigma$ را بزرگ می‌کند که باید آن را لو بدهد: این **پوشاندن** (masking) است.

### مثال حل‌شده‌ی ۳: یک جهش در هشت نمونه

هفت خوانش شتاب نزدیک صفر و یک جهش 25 m/s² می‌دهد $\bar{x} = 3.16$، $\sigma = 8.26$ و $z = 2.645 < 3$: علامت نمی‌خورد و روی کران $\sqrt{7} = 2.6458$ است. جایگزین مقاوم، میانه و MAD (میانه‌ی قدرمطلق انحراف) را به‌جای میانگین و انحراف معیار

می‌گذارد: میانه 0.05، MAD برابر 0.15 و امتیاز مقاوم $(25 - 0.05)/(1.4826 \cdot 0.15) \approx 112$. علامت می‌خورد.

مقاوم بودن کافی نیست وقتی قاعده **سراسری** است. حصارهای توکی $[Q_1 - kR, Q_3 + kR]$ با $R = Q_3 - Q_1$ مقاوم‌اند، اما روی کل لاگ ۲۵۰ نمونه‌ی شتاب را علامت می‌زنند که ۲۳۹ تای آن‌ها ترمز واقعی است. **فیلتر Hampel** هر نمونه را با میانه و MAD پنجره‌ی ۷ نمونه‌ای خودش می‌سنجد، اگر $|x_i - \tilde{x}| > 3 \cdot 1.4826 \cdot \mathrm{MAD}$ باشد علامتش می‌زند و میانه‌ی پنجره را جایش می‌گذارد: ۱۱۷ علامت، همه نزدیک جهش‌ها یا پژواک‌های گردوغبار. پژواک 1.2 متری وقتی دیوار در ۸ متری است، به‌خودی‌خود فاصله‌ای معتبر است؛ فقط همسایه‌ها نشان می‌دهند غلط است. اثر جانبی را هم گزارش کنید: فیلتر نمونه‌های نویزی را حدود ۴ سانتی‌متر (فاصله) و 0.3 m/s² (شتاب) جابه‌جا می‌کند. مسئله‌ی ۱ کلاس همین را روی پژواک گردوغبار در هفت خوانش LiDAR تکرار می‌کند، با $z = -\sqrt{6}$ (باز هم بیشترین مقدار ممکن) و امتیاز مقاوم ۲۲۹.

## ۵. مرز اجراها را رعایت کنید

لاگ از ۱۶ رانندگی جدا تشکیل شده که پشت هم چسبیده‌اند. درون‌یابی، پنجره‌ی Hampel، میانگین متحرک و جابه‌جایی هدف همه به همسایه نگاه می‌کنند و باید در پایان اجرا متوقف شوند؛ وگرنه ده هدف آخر اجرای ۳ از اجرای ۴ می‌آید. `PerGroup(stage, group_column="run_id")` یک **Composite** است: خودش مرحله است و مرحله‌ای را در بر دارد و آن را جداگانه روی هر اجرا اعمال می‌کند. فقط مرحله‌های بدون حالت را می‌پذیرد (مقیاس‌گذار جدا برای هر اجرا یعنی مدلی دیگر) و با پیکربندی JSON هفته‌ی ۳ رفت‌وبرگشت دارد. دستورالعمل کلاس `rover_cleaning_pipeline()`: `SortAndDeduplicate` (از ۲۵۳۳ به ۲۵۰۸ سطر)، `MarkInvalid` روی فاصله (۵۳۰ خانه به NaN)، `PerGroup(HampelFilter)` روی فاصله و شتاب (۱۶۷ خانه تغییر)، `PerGroup(FillGaps)` روی فاصله و سرعت (۵۸ خانه پر شد)، `PerGroup(ForecastTarget)` (`range_ahead_m`، ۱۰ سطر بعد) و `DropNonFinite` (۱۸۵۹ سطر با همه‌ی ویژگی‌ها و هدف).

هیچ چیز در این دستورالعمل از داده آموخته نمی‌شود، پس اجرای آن پیش از تقسیم امن است. بیشتر ۶۴۹ سطر حذف‌شده ابتدای MNAR هر اجراست: دامنه‌ی مدل «دیوار در برد حسگر» است. بارگذار کارخانه‌ای `rover_range_forecast` (الگوی Factory هفته‌ی ۴) دستورالعمل را اجرا می‌کند و `Dataset`ی برمی‌گرداند که فیلد `groups` آن شماره‌ی اجراست.

## ۶. مقیاس‌بندی و یک قاعده

امتیاز z یعنی $(x - \mu)/\sigma$ وقتی داده‌ی پرت $\sigma$ را بزرگ کند آسیب می‌بیند؛ کمینه-بیشینه $(x - \min)/(\max - \min)$ اجازه می‌دهد یک پرت بقیه را فشرده کند؛ مقیاس مقاوم $(x - \mathrm{median})/\mathrm{IQR}$ فقط روی ستون ثابت شکست می‌خورد.

### مثال حل‌شده‌ی ۴: نزدیک‌ترین همسایه کدام است؟

پرسش در حرکت یکنواخت $q = (5.0\ \mathrm{m}, 0.0\ \mathrm{m/s^2})$، نمونه‌ی ترمز $A = (5.1, -1.0)$ و نمونه‌ی یکنواخت $B = (6.2, 0.0)$. فاصله‌های خام: $d_A = 1.005$ و $d_B = 1.200$، پس k-NN نمونه‌ی ترمز را کپی می‌کند. با تقسیم بر انحراف معیار آموزش (3.0 m و 0.4 m/s²): $d_A = \sqrt{(0.1/3)^2 + (1/0.4)^2} = 2.500$ و $d_B = 1.2/3 = 0.400$. مدل‌های فاصله‌محور و کاهش گرادیان به مقیاس‌بندی نیاز دارند؛ درخت تصمیم (هفته‌ی ۷) نه.

**قاعده:** هر جایگذار، برش‌دهنده و مقیاس‌گذار *عددی می‌آموزد*. آن را فقط از سطرهای آموزش بیاموزید و همان عدد را روی سطرهای آزمون اعمال کنید. `ColumnStatStage` این را با متد قالبی تضمین می‌کند: `fit` تابع `learn()` را صدا می‌زند و `stats_` را ذخیره می‌کند، `transform` تابع `apply()` را صدا می‌زند و اگر `fit` اجرا نشده باشد `NotFittedError`

می‌دهد. مقدار آزمون بیرون از $[0, 1]$ پس از کمینه-بیشینه یک یافته است (رانندگی آزمون به جایی رفته که آموزش نرفته)، نه باگ.

## ۷. نشت داده و تقسیم صادقانه

نشت یعنی مجموعه‌ی آزمون واقعاً دیده‌نشده نیست (GeeksforGeeks علت‌ها را برمی‌شمارد: تقسیم نادرست، پیش‌پردازش پیش از تقسیم، ویژگی‌هایی که هنگام پیش‌بینی در دسترس نیستند و خطای خط لوله). برای سری زمانی بدترین حالت **تقسیم تصادفی سطرها**ست: با ۱۰ هرتز، هر سطر آزمون دوقلوهای آموزشی در فاصله‌ی 0.1 ثانیه دارد. با ۲۵٪ سطر آزمون، احتمال اینکه سطر آزمون دست‌کم یک همسایه‌ی آموزشی داشته باشد $1 - 0.25^2 = 0.9375$ است. همان مدل، چهار روش تقسیم:

| تقسیم | RMSE k-NN [m] | RMSE خطی [m] | `check_split` |
|---|---|---|---|
| سطرهای تصادفی | 0.319 | 0.098 | LEAKY |
| زمانی | 0.634 | 0.120 | LEAKY (یک اجرا دو نیم شده) |
| زمانی با فاصله‌ی ۱۰ سطری | 0.634 | 0.120 | OK |
| بر اساس اجرا (`GroupHoldoutSplit`) | **0.641** | **0.121** | OK |

تقسیم تصادفی خطای k-NN را نصف می‌کند چون k-NN همسایه‌ها را حفظ می‌کند؛ مدل خطی چهار وزن دارد و تقریباً متوجه نمی‌شود. اگر $k$ را روی تقسیم تصادفی تنظیم کنید، 0.32 m گزارش می‌دهید و 0.64 m تحویل. `check_split` سطرهای مشترک، اجراهای مشترک و کمترین فاصله‌ی آموزش و آزمون را در برابر افق گزارش می‌دهد. میانگین متحرک متقارن نشتی آرام‌تر است (نمونه‌های آینده را میانگین می‌گیرد)؛ `CausalRollingMean` فقط از گذشته استفاده می‌کند و `ExpandingWindowCV` تاخوردگی‌های زمان‌مرتب برای جست‌وجوی شبکه‌ای هفته‌ی ۷ فراهم می‌کند.

## ۸. گردش کار بدون نشت

پاک‌سازی بدون حالت روی کل لاگ، هر بار یک اجرا؛ تقسیم بر اساس اجرا (یا زمان، با فاصله‌ای دست‌کم برابر افق)؛ برازش مرحله‌های آموختنی (جایگذاری، برش، مقیاس) روی سطرهای آموزش؛ تبدیل هر دو طرف با همان خط لوله‌ی برازش‌شده؛ برازش و ارزیابی مدل و گزارش روش تقسیم. نتیجه روی لاگ کلاس، بدون هیچ تغییری در مدل‌ها: مدل پایه‌ی میانگین 3.833 m، خطی **0.121 m** (از 2.73 m) و k-NN برابر 0.641 m. وزن‌های مدل خطی فیزیک را بازمی‌یابند، $r(t+1) \approx r - v \cdot 1\,\mathrm{s}$ به‌علاوه‌ی جمله‌ی کوچک شتاب؛ وزن نرخ گردش نزدیک صفر است و به همین دلیل جایگذاری میانه برای بخش‌های یخ‌زده‌ی آن پذیرفتنی است.

**جمع‌بندی.** اکنون می‌توانیم لاگ خام را ممیزی کنیم، سازوکار گمشدگی را تشخیص دهیم و راه‌حلش را انتخاب کنیم، جهش را با قاعده‌های محلی و مقاوم از مانور جدا کنیم، گام‌های همسایه‌محور را درون یک اجرا نگه داریم، فقط با آماره‌های آموزش مقیاس‌بندی کنیم، و داده‌ی زمانی و گروهی را بدون نشت تقسیم کنیم و آن را اثبات کنیم. تکلیف ۲ همه‌ی این‌ها را روی لاگی تازه (بذر ۵۲) به کار می‌گیرد و یک آشکارساز حسگر یخ‌زده، یک برش‌دهنده‌ی آموختنی توکی و تابع بدون نشت `prepare()` اضافه می‌کند.